Google has two customized FMSs: GFS, described in the chapter, and a newer file system, called BigTable, that supports applications such as Google Earth and Google Finance. Investigate BigTable to determine how it's implemented and what application services it provides. What are the unique characteristics of applications using BigTable, and how is it optimized for these applications?

BigTable vs. SQL

BigTable is a NoSQL database created by Google as a way to fill in the gap between large relational databases like .sql files and messy/unorganized data set files like comma separated values (CSVs). Relational databases like SQL databases provide great querying and indexing and have countless use cases, but they struggle with horizontal expansion (expanding to petabytes of data) across many machines. Of the biggest reasons relational databases are slow across machines is joins. The reason joining tables in a single machine is so fast is because the data is local to that machine. When joining tables across separate machines, the data has to be transmitted across the network which compounds very quickly into a major time constraint. The way BigTable addresses this issue is by eliminating joins altogether. This comes as a disadvantage of denormalized data (meaning the data can and is likely to be duplicated across multiple rows). This is a deliberate compromise made by the Google developers, however, as it allows BigTable tables to be enormous in size and the data spread across many machines.

The second reason relational databases don't scale well is because of how rigid and strictly defined the column structure must be. All columns in (for example) a SQL database must be defined. All records entered must align exactly with what the column accepts. BigTable, however, does not require such rigid column structure. It uses a sparse and flexible column structure that allows new columns to be added without

needing to define every accepted data type and size, reducing the logistical planning and cost of expansion to almost nothing.
Yet another reason BigTable is much more scalable than relational databases is that BigTable tables are split into multiple tablets in a process known as sharding. These tablets are then spread across tablet servers where they are processed parallelly. This parallel processing makes read and write operations much faster than relational databases.

CSV Type Files

The problems that are associated with list formats like CSVs are solved by BigTable in a more obvious way. Firstly, there is no indexing in CSV files, meaning you can't search for something without doing a top-down search (checking if every single value in the file matches what you're looking for). BigTable solves this with row-keys. Row-keys are sorted lexicographically, meaning you can jump to any record or range of recording without the overhead of scanning everything in the file. The second issue that CSVs or similar files share that BigTable solves is lack of versioning. If a record in a CSV is overridden, that record no longer exists. With BigTable, records are timestamped exactly with versions of cells being easily queried.

BigTable Querying API

BigTable has a native API that allows for basic querying of data. It is not nearly as robust as SQL in terms of what querying commands are available, but these limitations actually forced data engineers to think about data access patterns proactively as opposed to writing complex SQL queries after the development of the schema. This made data design much more efficient, and in turn made horizontal scaling extremely cheap. The querying API allows for the following commands in BigTable:

Single Row Lookup	Given a row key, able to instantly fetch the data for that row.
Range Scan	Given a start and end row, able to instantly fetch the data of the rows in the given range.
Single Row Write	Inserts data in a cell given a row key, column family, column qualifier, and timestamp.
Delete	Mark a cell, column, or row for deletion. The marked data is deleted at the next compaction cycle.

BigTable Applications

BigTabel was developed by Google and optimized for Google applications. Arguably the most efficient use of BigTable is Google Earth. Geographical data can be represented with coordinates, where rows adjacent to each other map to adjacent geographical locations. This means that when a user is on google street view or simply viewing any given area on Google maps, a BigTable range scan can be used to fetch all the data of the location nearby where the user is. This makes getting the data on Google Earth very quick which leads to a seamless experience for the user.

Google Finance makes perfect use of BigTable's timestamps feature. Financial records are almost entirely dependent on time. In other words, data like stock market prices, foreign exchange conversion rates, transaction history, etc. cannot be represented well without constantly being updated to reflect the most accurate (and timely) figures. As we've previously mentioned, BigTable records are timestamped with the precise time down to the microsecond, and are able to be queried by their timestamps. This allows for accurate retrieval of time-sensitive records like the financial data Google Finance uses. The strong versioning that BigTable provides also allows for accurate and efficient backlogs and histories of finance data that can be used for analysis.